CONTINUED

2 Complete the truth table for the given logic circuit in Figure 10.13:

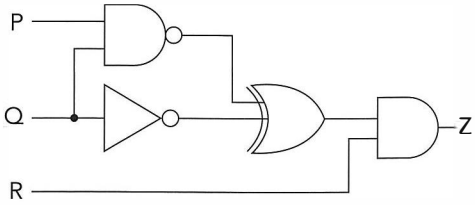


Figure 10.13: Complete the truth table for this logic circuit

P	Q	R	Workspace	Z
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

10.8 Logic expressions

You need to be able to write a logic expression for a logic circuit. The first thing you need to do to be able to write this is to break down the different parts of the logic circuit. Let's look at the logic circuit in Figure 10.14 as an example:

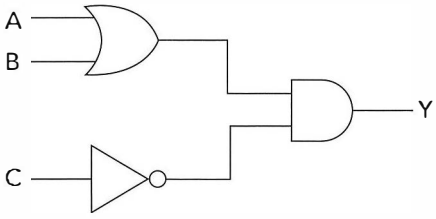


Figure 10.14: A simple logic circuit

The first part to the logic circuit is the OR gate. The logic expression for this part is A OR B. The second part of this logic circuit is NOT and the third part of this logic circuit in the AND gate.

So, the three parts to this logic circuit are:

A OR B

NOT C

AND

You can now start to join these different parts together to create the complete logic expression. You should always start with the label for the final output followed by an equals symbol (=):

Y =

One of the simplest ways to complete the expression is to then add the final gate in the logic circuit as everything else is built around this:

Y = AND

Then add the part that creates the first input to this gate. You should put this into brackets to show that it is a different part of the logic expression:

Y = (A OR B) AND

Then add the part that creates the second input to this gate to create the complete logic expression:

Y = (A OR B) AND (NOT C)

Let's look at a larger logic circuit (Figure 10.15) as a second example:

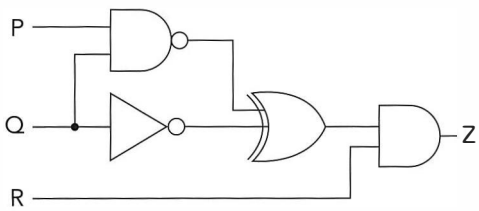


Figure 10.15: A larger logic circuit

First, break down the logic circuit into its different parts:

P NAND Q

NOT Q

XOR

AND R

Then join together these different parts to create the logic expression.
Remember, a simple way to do this is to start with the final logic gate and work from right to left.

Z = AND R

Z = (XOR) AND R

Z = ((P NAND Q) XOR) AND R

Z = ((P NAND Q) XOR (NOT Q)) AND R

You now know how to write a logic expression for a logic circuit, that's awesome!

Questions

19 Write the logic expression for the given logic circuit:

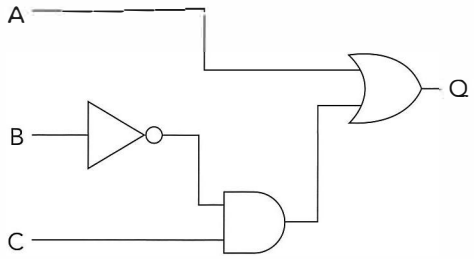


Figure 10.16: Find the logic expression for this circuit

20 Write the logic expression for the given logic circuit:

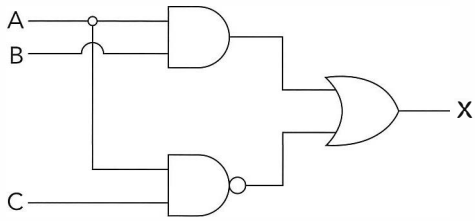


Figure 10.17: Find the logic expression for this circuit

10.9 Representing truth tables

You have learnt how to complete a truth table for a logic expression and a logic circuit. You now need to learn how to write a logic expression and draw a logic circuit from a truth table.

Let's start with a simple truth table that has two input values A and B:

A	B	X
0	0	**0**
0	1	**1**
1	0	**0**
1	1	**0**

If you look at this truth table, the first thing you should ask yourself if whether you know any single logic gates that would produce this truth table. It doesn't match any of the truth tables that you have learnt previously in this chapter for each of the logic gates, therefore you now know it must be a combination of at least two logic gates.

When you are creating a logic expression and a logic circuit from a truth table, you only need to know how to do this using three of the logic gates that you have learnt: the NOT gate, the AND gate and the OR gate.

Once you have looked at the truth table to see if it represents a single logic gate that you know, if you find it doesn't, you can move onto the next step which is finding out which gates will create that truth table.

Looking at the truth table again:

A	B	X
0	0	**0**
0	1	**1**
1	0	**0**
1	1	**0**

It has one output value that is a 1, so this is what you need to concentrate on. You need to use an AND gate to create this output. At present an input value of A = 0 and B = 1 will not result in an output of 1 using an AND gate. You should remember that for the output to be 1 for an AND gate, both inputs need to be 1. Therefore, you need to reverse input A, which is 0, to make it 1. You should remember that to reverse the input value to create the opposite value as an output, you should use a NOT gate. That makes the logic expression for this truth table:

X = (NOT A AND B)

You can then draw a logic circuit for this (Figure 10.18):

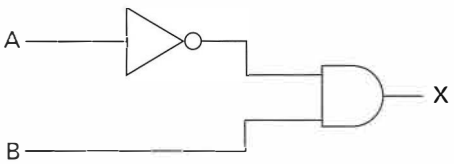


Figure 10.18: The logic circuit for logic expression X = (NOT A AND B)

Let's look at another truth table:

A	B	X
0	0	**0**
0	1	**1**
1	0	**0**
1	1	**1**

This time the truth table has two output values that are 1, so you need to focus on both of those. You have the same situation again with the first output of 1, so you already know that this part is NOT A AND B.

A	B	X
0	0	**0**
0	1	**1**
1	0	**0**
1	1	**1**

← NOT A AND B

You now need to look at the second output value of 1. You need to think whether an input of 1 for A and an input of 1 for B will create an output of 1 if an AND gate is used. You should recognise that it will. This means that this part is A AND B.

A	B	X	
0	0	**0**	
0	1	**1**	← NOT A AND B
1	0	**0**	
1	1	**1**	← A AND B

Now that you have worked out what the part of the logic expression is for each output value of 1, you need to join these together using an OR gate. That makes the logic expression for this truth table:

X = (NOT A AND B) OR (A AND B)

You can then draw a logic circuit for this (Figure 10.19):

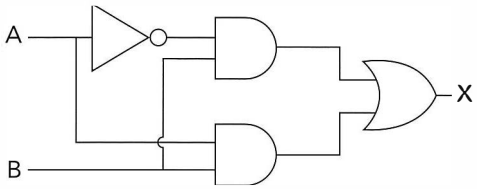


Figure 10.19: The logic circuit for logic expression X = (NOT A AND B) OR (A AND B)

Let's now look at a truth table that has three input values A, B and C:

A	B	C	X
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**0**
1	0	0	**0**
1	0	1	**1**
1	1	0	**0**
1	1	1	**1**

This makes the process more complex and there are several methods that you can use to work out the logic expression. Each method may generate a slightly different logic expression, but they all should produce the same truth table. It is the structure of them that will change slightly, but the logic of them will ultimately produce the same result.

The first method you can use is a method called sum of products. In this method you need to create a sum of products expression for each row in the table that has an output value of 1. In a sum of products statement, you need to look at each input value. If it is 1, then in the expression it will be, for example, A. If it is 0, then in the expression it will be, for example, NOT A. Each input is then summed, this is done using the AND gate.

Let's look at the first row with an output of 1 in the table:

A	B	C	X
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**0**
1	0	0	**0**
1	0	1	**1**
1	1	0	**0**
1	1	1	**1**

In this row:

- A is 0 so that's NOT A.
- B is 0 so that's NOT B.
- C is 1 so that's C.

These are then joined using AND gates making:

NOT A AND NOT B AND C

You need to follow this process for each row in the truth table that has an output of 1. You should see this result when you do:

A	B	C	X	
0	0	0	**0**	
0	0	1	**1**	← NOT A AND NOT B AND C
0	1	0	**0**	
0	1	1	**0**	
1	0	0	**0**	
1	0	1	**1**	← A AND NOT B AND C
1	1	0	**0**	
1	1	1	**1**	← A AND B AND C

Each of the sum of product expressions is then joined together with an OR gate creating the final expression:

X = (NOT A AND NOT B AND C) OR (A AND NOT B AND C) OR (A AND B AND C)

A person would normally look to then simplify this statement, but you do not need to understand how to do this. You could research how to do this though if you wanted to give yourself a challenge.

The second method you could use looks for the constant values in each row where the output is 1.

Let's look at another truth table:

A	B	C	X
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**1**
1	0	0	**0**
1	0	1	**0**
1	1	0	**1**
1	1	1	**1**

Again, you look at the rows that have an output of 1. If you look at the first row with an output of 1, the input values are A = 0, B = 0, C = 1. You need to look to see if any of these values are constant. This means, are there any values that are the same between the inputs. You should notice that the input values for A and B are the same, 0 and 0.

You now need to look whether inputs of 0 and 0 for A and B always result in an output of 1 in each row in the table. If they do, this means that the expression for this row is:

NOT A AND NOT B

If they don't always result in an output of 1, then you use the sum of products expression for the row instead. If you look at row 1 in the table, you should notice that A and B both have an input of 0, but that the output is 0 and not 1. Therefore, you need to use the sum of products expression for row 2 instead, which is:

NOT A AND NOT B AND C

This is because we need to apply the rule you learnt earlier about reversing the inputs to make sure that the values will give an output of 1 for an AND gate.

If you look at the next row that has an output of 1, you should see that the inputs are A = 0, B = 1, C = 1. This time it is B and C that have the constant value as they are both the same. You now need to look at whether all rows in the table where B and C have the value 1, the result is an output of 1. You should be able to notice that the only other place in the table where the inputs are both 1 is row 8, the final row. This does result in an output of 1 as well, so this means that the expression for this row is:

A AND C

If you look at the final two rows in the table, they both that have an output of 1. If you have two outputs of 1 together like this, you can treat them as a pair. You apply the same method to see if there are any values that are constant. In row 7 the values are A = 1, B = 1 and C = 0, and in row 8 the values are A = 1, B = 1 and C = 1. You should notice that the A and B inputs are constant in both of these rows. You now look to see if there are any other instances of A and B both having an input of one in the table to see if this results in an output of 1 also. There are no other instances, so this means the expression for these two rows is:

A AND B

These three expressions are then joined together again with an OR gate, giving a final expression of:

X = (NOT A AND NOT B AND C) OR (A AND C) OR (A AND B)

If you work out the statement using the sum of products method, you will get:

X = (NOT A AND NOT B AND C) OR (NOT A AND B AND C) OR
(A AND B AND NOT C) OR (A AND B AND C)

The two statements represent the same truth table, but you can see that the second method creates a shorter statement, which can be helpful if there are lots of outputs of 1 to be considered.

Question

21 Look at the given truth table:

A	B	X
0	0	1
0	1	1
1	0	0
1	1	0

a Write a logic expression to represent the given truth table.

b Draw a logic circuit to represent the given truth table.

10.10 Representing logic expressions

You need to be able to look at a logic expression and create a logic circuit and a truth table from the expression.

Let's look at a simple logic expression:

Z = (R AND S) OR (R AND T)

You should notice that this logic expression has:

- 3 input values: R, S and T
- an output value of Z
- uses the logic gates AND and OR.

If you look at the brackets you can see that the expression has two main parts that are joined together by an OR gate. You should learn from this that the OR gate will be the final gate in the logic circuit. You can the draw this part of the circuit (Figure 10.20):

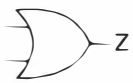


Figure 10.20: The logic circuit for the expression Z = (something) OR (something)

You the need to look at the parts of the expression that are in the brackets. Each bracketed section is an input into the OR gate that you have drawn. This means that the first input to the OR is the output of R AND S (Figure 10.21):

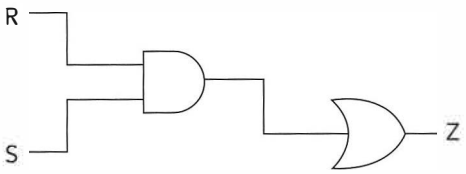


Figure 10.21: The logic circuit for the expression Z = (R AND S) OR (something)

The second input to the OR is the output of R AND T (Figure 10.22):

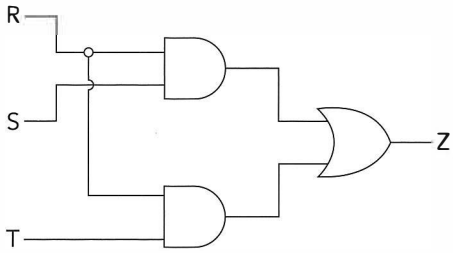


Figure 10.22: The logic circuit for the expression Z = (R AND S) OR (R AND T)

You now have a logic circuit that represents the logic expression!

You can now use what you have previously learnt to represent that as a truth table:

R	S	T	Z
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**
0	1	1	**0**
1	0	0	**0**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

Let's look at another logic expression:

X = ((A AND B) OR NOT C) XOR (B NOR C)

You should notice that this logic expression has:

- 3 input values: A, B and C
- an output value of X
- uses the logic gates AND, OR, NOT, XOR and NOR.

You can use the same method as before to structure the logic circuit. You should be able to see again that the expression has two main bracketed sections. You will also notice this time that one of those bracketed sections has brackets within it as well. First, you should notice that the final logic gate in the logic circuit will be an XOR gate. You can now begin to draw the circuit (Figure 10.23):

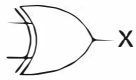


Figure 10.23: The logic circuit for the expression X = (something) XOR (something)

If you look at the larger bracketed area to the left of the XOR in the statement you can begin to break this down. This will become the first input for the XOR gate. The bracketed section within this is A AND B. The brackets around this mean that the output of A AND B needs to become the input of the remaining part of this whole section. The remaining part is OR NOT C. What you should learn from this is that the output on A AND B will become the input to an OR gate and that the second input to the OR gate will be NOT C. You can now continue to draw the circuit (Figure 10.24):

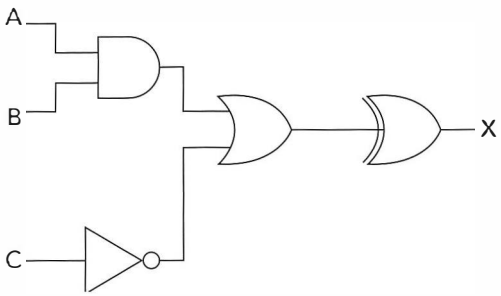


Figure 10.24: The logic circuit for the expression X = ((A AND B) OR NOT C) XOR (something)

You now need to look at the final bracketed section as this will create the second input for the XOR gate. This is B NOR C, so you can now complete the logic circuit (Figure 10.25):

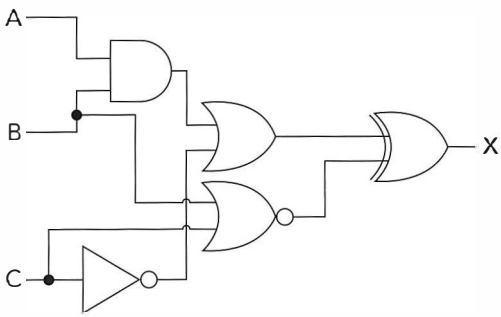


Figure 10.25: The logic circuit for the expression X = ((A AND B) OR NOT C) XOR (B NOR C)

Questions

22 Complete the truth table for the logic statement:
X = ((A AND B) OR NOT C) XOR (B NOR C)

A	B	C	X
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

23 Draw a logic circuit to represent the logic expression:
Z = (A OR B) NAND ((C AND B) OR NOT A))

10.11 Representing problem statements

The final thing you need to be able to do is to write a logic expression, draw a logic circuit and complete a truth table for a problem statement.

A problem statement is when a logic problem is described in natural language and you are given criteria for the logic of the problem within that description.

An example of a problem statement is:

A factory manufactures soap and has a system that monitors the manufacturing process. The system has three conditions to monitor the acidity (A) the temperature (T) and the pressure (P).

The conditions for the system are:

Input	Binary value	Condition
A	1	pH <= 7
	0	pH > 7
T	1	Temperature < 50 °C
	0	Temperature >= 50 °C
P	1	Pressure >= 75%
	0	Pressure < 75%

The system will sound an alarm (X) when certain conditions are detected. The alarm will sound when:

the pressure >= 75% and the temperature >= 50 °C

or

the temperature < 50 °C and the pH > 7

You can begin to break down this problem and write a logic expression for it. You can take the alarm conditions line by line and break them down to create each part of the expression.

Let's look at the first line of the alarm condition:

The pressure >= 75% and the temperature >= 50 °C

You should notice this involves the inputs P and T. You need to work out what the binary value for the input is for each of these conditions. You can do this by looking at the table you are given. You should work out that P = 1 and T = 0.

This line of the problem statement states that if both of those conditions happen, the alarm will sound. This means that the expression would use an AND gate and that both inputs must be 1 going into that AND gate in order to create an output of 1 to sound the alarm. As T = 0, this value will need to be reversed and you should remember that you can use a NOT gate to do this. This means that the expression for this part of the problem statement is:

P AND NOT T

The second line of the alarm condition is simply an OR. You should learn from this that the two sets of conditions will be separated using and OR gate. This means that expression now becomes:

(P AND NOT T) OR

The third line of the alarm conditions is:

The temperature < 50 °C and the pH > 7

You should notice this involves the inputs T and A. You again need to look at the table to work out the binary value for the input for each condition. You should work out that T = 1 and A = 0. It states that both of the conditions needs to be met for the alarm to sound, so the expression for this part of the problem is:

T AND NOT A

You can now add this to the other part and create the full logic expression for the problem statement, which is:

X = (P AND NOT T) OR (T AND NOT A)

Using what you have previously learnt, you can create the logic circuit and the truth table from this to represent the problem statement (Figure 10.26):

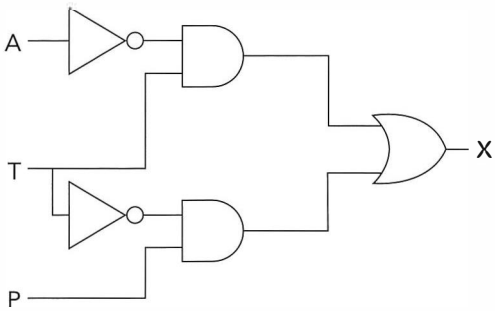


Figure 10.26: The logic circuit for the soap factory problem statement

A	T	P	X
0	0	0	**0**
0	0	1	**1**
0	1	0	**1**
0	1	1	**1**
1	0	0	**0**
1	0	1	**1**
1	1	0	**0**
1	1	1	**0**

REFLECTION

Which type of Boolean logic task do you prefer to work out?

For example, do you prefer to draw a logic circuit for a logic expression, do you prefer to complete a truth table for a logic circuit?

Why do you prefer that Boolean logic task?

COMPUTER SCIENCE IN CONTEXT

As you can see from studying problem statements, many systems use more than one sensor to monitor processes such as manufacturing. The data from each of these sensors provides an input into a logic circuit to determine what the output will be. Some manufacturing business use as many as ten different sensors in a logic circuit to monitor their manufacturing processes. That would be one huge logic circuit!

SUMMARY

A logic gate is a component in a computer that controls the flow of electricity.
A single value is output from a logic gate, this can be a 1 or 0. 1 sets it to high, 0 sets it to low.
A problem statement is a description of the conditions of an electrical system.
A logic circuit is a diagrammatic representation of an electrical system showing the logic gates it uses. The logic gates it can use are NOT, AND, OR, NAND, NOR and XOR.
A truth table is a representation of all the different outputs of an electrical system, dependent on the values of each input.
A logic expression is a mathematical representation of an electrical system.

EXAM-STYLE QUESTIONS

1 **State** the purpose of a logic gate in a computer system. **[1]**

2 Draw the single logic gate that would generate the given truth tables:

a

A	B	X
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

[1]

b

A	B	X
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**0**

[1]

[Total: 2]

COMMAND WORD

state: express in clear terms.